Sprint 5 · Reto «¿La ocupación depende de esto?»

Ficha completa del reto, para consultar. Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO **Ficha del reto**

1 Qué os pide el cliente

Qué os pide el cliente. Al dueño de TurisData le han dicho dos cosas: *«cuando hace calor se llena menos»* y *«los festivos llenan»*. Antes de cambiar personal y precios, quiere saber **cuánto** cambia realmente la ocupación diaria con la temperatura y con los festivos, y **con qué seguridad** se puede afirmar. Os da la serie diaria de **tres años** (1096 días). Vuestra respuesta es un **cuaderno** con el análisis, una **base de datos** con los datos y una **respuesta al cliente de 150 palabras como máximo**.

Cómo se trabaja. Es el reto más «de números» hasta ahora, pero no hace falta ser matemático: os damos un **cuaderno plantilla** en Colab con las celdas por rellenar y unos **puntos de control** (`assert`) que os dicen al momento si lo que habéis escrito es correcto. Lo más importante: el **descenso del gradiente** (el método con el que un modelo «aprende» ajustando números poco a poco) lo escribís vosotros, sin usar la función ya hecha de una librería. En clase tenéis 10 minutos de planning (día 1) y, el día 2, la práctica de CRUD (que es el paso 1), **45 minutos de reto + 40 de entrega, review y retro**; lo que no quepa va a casa. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
El cuaderno plantilla `S05_07_reto_ocupacion`	Es el entregable: lo rellenáis celda a celda. Las celdas con `...` o `# TODO` son vuestras	Colab. Usad *Archivo → Guardar una copia en Drive* y compartidla con el equipo (permiso de edición)
Los datos `ocupacion_diaria.csv`	1096 días, **sintéticos**: `fecha`, `ocupacion_pct`, `temperatura_c`, `festivo` (1 = festivo) y `precio_medio`	El cuaderno los descarga solo con la función `dato("ocupacion_diaria.csv")`
Python y sus bibliotecas: `sqlite3`, `pandas`, `numpy`, `matplotlib`, `scipy`, `statsmodels`	Base de datos, tablas, gráficos, regresión y contrastes	Colab las trae instaladas; el cuaderno las importa en su celda 3
Cuadernos del sprint que son la base del reto: `S05_03` (gradiente), `S05_04` (intervalos de confianza), `S05_05` (contrastes y `statsmodels`) y `S05_06` (CRUD, que es el paso 1)	Las pistas y el código parecido que podéis reutilizar	Colab. Su código explicado y el de `S05_07`
Presentaciones y apuntes	Repasar el gradiente (S05.2), la estadística y el CRUD (S05.3)	S05.2 · S05.3 · apuntes
Cómo entregar en GitHub	Subir los ficheros a la carpeta `sprint-05/`	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender ideas y errores y pedir pistas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Guía paso a paso

Son seis pasos, de **105 minutos en total**. Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. En el horario del día 2 caben unos 70 minutos (la práctica de CRUD, que es el paso 1, y 45 de reto); los 35 que faltan se hacen en la **consolidación** (13:00–13:30) y en la **6.ª hora** (hasta las 14:30),

y la subida puede hacerse hasta esa hora. Completad el nivel mínimo y dejad lo demás para casa. Todo se hace **dentro del cuaderno**, de arriba abajo; ejecutad las celdas en orden.

Paso 1 · Cargar los datos en una base de datos (día 2, 30 minutos)

1. Abrir el cuaderno `S05_07` en Colab, guardar una copia y compartirla con el equipo.
2. Ejecutar las dos primeras celdas de código (preparación e importaciones).
3. Crear `turisdata_reto.db` con la tabla `ocupacion_diaria` y un índice, y leerla con SQL.

Cómo se hace

- **Preparar el cuaderno.** Abrid el cuaderno, pulsad *Archivo → Guardar una copia en Drive* y compartidla con el equipo. Una persona escribe y las demás miran y opinan; cambiad de turno cada pocos minutos. Ejecutad la celda de «Preparación» y la de importaciones.
- **Cargar el CSV.** La función `dato("ocupacion_diaria.csv")` os da la ruta del fichero. Leedlo con `pd.read_csv`.
- **Crear la base.** Abrid una conexión con `sqlite3.connect(RUTA_BD)` y llamadla **`con`** (los puntos de control usan ese nombre). Guardad la tabla con `to_sql("ocupacion_diaria", con, if_exists="replace", index=False)`. Sin `replace`, al repetir la celda os daría error o duplicaría datos.
- **Crear el índice.** Con `con.execute`, la orden `CREATE INDEX IF NOT EXISTS idx_ocupacion_fecha ON ocupacion_diaria(fecha)` y después `con.commit()`. El nombre del índice tiene que ser exactamente ese.
- **Leer con SQL.** `pd.read_sql_query(...)` con una consulta `SELECT ..., CAST(SUBSTR(fecha, 6, 2) AS INTEGER) AS mes FROM ocupacion_diaria`. El resultado se llama **`diaria`** y trae, además de las columnas del CSV, la columna `mes` (un número del 1 al 12). Este paso es la práctica de CRUD: si ya hicisteis `S05_06`, copiad el código.

Qué deberíais ver. El mensaje `Punto de control 1 superado`, y en el panel de archivos de Colab el fichero `turisdata_reto.db`. Si falla, el mensaje del `assert` dice qué falta (el archivo, las 1096 filas, la columna `mes` o el índice).

Hecho cuando: `turisdata_reto.db` existe, `diaria` tiene 1096 filas con la columna `mes` y el punto de control 1 está superado.

Paso 2 · Mirar los datos (día 2, 10 minutos)

1. Describir la serie y calcular la media de ocupación por mes.
2. Hacer al menos dos gráficos: ocupación por mes y ocupación frente a temperatura.
3. Escribir 2–3 observaciones con vuestras palabras.

Cómo se hace

- **Describir.** `diaria.describe()` muestra la media, la dispersión, el mínimo y el máximo. La media por mes sale con `diaria.groupby("mes")["ocupacion_pct"].mean()`.
- **Dos gráficos** con `matplotlib`: un diagrama de caja por mes (`boxplot`) y una nube de puntos de temperatura contra ocupación (`scatter`). Todos con **título y ejes en español** y con las unidades (`°C`, `%`).
- **Observaciones.** En la celda de texto «Mis observaciones» escribid 2–3 frases: ¿en qué meses hay más y menos ocupación? ¿Se ve una relación con la temperatura? ¿Coincide el calor con unos meses concretos? Es la base de vuestra respuesta al cliente.

Qué deberíais ver. Una ocupación alta en invierno (en torno al 78–79 % de media en enero, febrero y diciembre) y más baja de mayo a octubre (en torno al 60–63 %), y una nube de puntos que **baja de izquierda a derecha**: a más temperatura, menos ocupación. No hay `assert` en este paso: lo revisa el equipo.

Hecho cuando: hay dos gráficos con título y ejes en español y 2–3 observaciones escritas.

Paso 3 · Regresión lineal desde cero (día 2, 35 minutos)

1. Escribir `gradiente_mse` y `entrenar` (el descenso del gradiente).
2. Ajustar el modelo simple (solo temperatura) y dibujar la curva de aprendizaje.
3. Ajustar el modelo con control (temperatura, festivo y mes).

Cómo se hace

- **La idea.** Si la ocupación fuera `a + b × temperatura`, queremos las `a` y `b` que dan la mejor recta. El **error** (MSE) mide lo mal que se ajusta; el **gradiente** indica en qué dirección cambiar `a` y `b` para que el error baje. El descenso del gradiente repite: calcular el gradiente, dar un pasito en la dirección contraria y volver a empezar. Si lo necesitáis, repasad `S05_03`: el ejemplo de una sola variable es el mismo.
- **`gradiente_mse(X, y, w)`** devuelve $(2/n) \cdot X^T(Xw - y)$. Con NumPy: `2 / len(y) * X.T @ (X @ w - y)`.
- **`entrenar(X, y, tasa, epocas)`** empieza con `w` a ceros y repite `epocas` veces: `w = w - tasa * gradiente_mse(X, y, w)`, guardando el MSE de cada vuelta en una lista `historial`. Devuelve `(w, historial)`. **Sin `scikit-learn`.**

- **Modelo simple.** Estandarizad la temperatura (restar la media y dividir entre la desviación) y añadid una columna de unos para la constante. Entrenad (probad con tasa 0,1 y unas 2000 épocas). Para dar la pendiente en **puntos por ºC**, deshaced la estandarización: dividid el coeficiente entre la desviación de la temperatura. Guardadla en `pendiente_simple`.
- **Curva de aprendizaje.** `plt.plot(historial)` con título y ejes en español, y una línea de texto debajo: ¿baja y se estabiliza? Si sube sin parar o sale `NaN`, la tasa es demasiado grande: reducidla.
- **Modelo con control.** Las variables son la temperatura, `festivo` y los indicadores de mes: `pd.get_dummies(diaria["mes"], drop_first=True)` da las 11 columnas. Estandarizad todas, entrenad (unas 2000 épocas, tasa 0,1) y dividid cada coeficiente entre la desviación de su variable para volver a las unidades originales. Guardad `efecto_temp_control` y `efecto_festivo_control`.

Qué deberíais ver. Una curva que baja rápido y se aplana, y el mensaje `Punto de control 2 superado: simple -1.86 · con control: temp -0.09, festivo +9.11`. Fijaos en que **la temperatura casi deja de importar** cuando se controla el mes. En el nivel mínimo, el punto de control dará error en las líneas del modelo con control hasta que lo hagáis.

Hecho cuando: `pendiente_simple` sale ≈ −1,86, la curva de aprendizaje está dibujada y comentada y (en el estándar) el punto de control 2 está superado.

Paso 4 · Comparar con una librería (día 2, 10 minutos)

1. Ajustar los mismos dos modelos con `statsmodels`.
2. Reunir los tres coeficientes en la tabla `comparacion`.
3. Comprobar que coinciden con los vuestros.

Cómo se hace

- **Ajustar.** `ols_simple = smf.ols("ocupacion_pct ~ temperatura_c", data=diaria).fit()` y `ols_control = smf.ols("ocupacion_pct ~ temperatura_c + festivo + C(mes)", data=diaria).fit()`. `C(mes)` le dice a la librería que `mes` son categorías.
- **La tabla.** Un DataFrame `comparacion` de **3 filas × 2 columnas**: las filas son `pendiente_simple`, `efecto_temp_control` y `efecto_festivo_control`; las columnas, `desde_cero` (vuestros valores) y `statsmodels` (los de `ols_simple.params["temperatura_c"]`, `ols_control.params["temperatura_c"]` y `ols_control.params["festivo"]`).
- **Si no coinciden**, la causa más común es una tasa de aprendizaje inadecuada o pocas épocas. Probad con más vueltas antes de buscar un fallo en el código.

Qué deberíais ver. `Punto de control 3 superado` y la tabla con dos columnas casi idénticas (se admite una diferencia de 0,02).

Hecho cuando: `comparacion` tiene 3 × 2 valores que coinciden y el punto de control 3 está superado.

Paso 5 · Medir la seguridad del resultado (día 2, 10 minutos)

1. Obtener el p-valor, el intervalo de confianza y el R^2.
2. Calcular el tamaño del efecto.
3. Explicar cada cifra con palabras.

Cómo se hace

- **Cuatro variables** del cuaderno: `p_temp_control = ols_control.pvalues["temperatura_c"]`; `ic_festivo = tuple(ols_control.conf_int().loc["festivo"])`; `p_festivo_ttest = stats.ttest_ind(festivos, no_festivos, equal_var=False).pvalue` (el t de Welch compara la ocupación de los días festivos con la de los demás; separad antes las dos series); `r2_control = ols_control.rsquared`.
- **El tamaño del efecto** es cuántos puntos de ocupación cambia el resultado: con un grado más, o con un festivo. Ya lo tenéis en los coeficientes.
- **Explicar con palabras.** Una frase por cifra, como en esta plantilla: *«Con un festivo, la ocupación sube X puntos (intervalo de confianza entre A y B)»*. El p-valor, en una frase: *«¿sería raro ver esto si no hubiera efecto?»*; si es muy pequeño, sí lo sería. Escribid las frases en una celda de texto, porque son la base de la respuesta.

Qué deberíais ver. `Punto de control 4 superado`, con estos valores aproximados: `p_temp_control` ≈ 0,48 (la temperatura **no** es significativa una vez controlado el mes), `ic_festivo` ≈ (7,35 ; 10,87), `p_festivo_ttest` minúsculo y `r2_control` ≈ 0,58.

Hecho cuando: el punto de control 4 está superado y cada cifra tiene su frase explicada con palabras.

Paso 6 · Responder al cliente en 150 palabras, reiniciar y revisar (día 2, 10 minutos)

1. Escribir la respuesta al cliente y contar las palabras.
2. Reiniciar el cuaderno y ejecutarlo todo de arriba abajo.
3. Pedir a alguien que no escribió esa parte que lo revise con la rúbrica y anotar el resultado.

Cómo se hace

- **La respuesta** contesta **cuatro cosas**, para una persona que no es técnica: (1) **qué encontráis**, con cifras e incertidumbre, para la temperatura y para los festivos; (2) **qué recomendáis**, algo que el cliente pueda hacer con personal o precios; (3) **qué no podéis afirmar** (correlación no es causalidad, el mes confunde, datos sintéticos de un solo alojamiento); (4) **una frase de IA responsable**, por ejemplo que el modelo no debería fijar precios sin revisión humana. Escribidla en la celda de texto de la sección 6. Contad las palabras (`len(texto.split())`); si pasa de 150, se recorta. **Toda cifra sale de vuestro cuaderno.**
- **Reiniciar.** En Colab: *Entorno de ejecución → Reiniciar y ejecutar todo*. Si algo falla, corregidlo antes de entregar. Marcad también la lista de la sección 7 del cuaderno.
- **Dos celdas de texto al final del cuaderno.** «Uso de IA»: qué pedisteis, qué comprobasteis y qué cambiasteis. «Revisión con la rúbrica»: copiad y rellenad esto, siguiendo los cinco pasos de «Cómo se revisa» de entrega y rúbrica:

```
## Revisión con la rúbrica
**Revisado por:** (nombre) · **Fecha:** (dd/mm/aaaa)

| Criterio | Nivel (1-4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** (una cosa concreta)
```

Si no cabe en los 10 minutos, la revisión se hace al empezar la entrega.

Qué deberíais ver. El cuaderno entero ejecutado sin errores y con los cuatro puntos de control superados, una respuesta de 150 palabras o menos y las dos celdas finales rellenas.

Hecho cuando: *Reiniciar y ejecutar todo* funciona sin errores, la respuesta cumple las cuatro preguntas y la «Revisión con la rúbrica» tiene un nivel y una frase de prueba por criterio.

Después, 40 minutos de entrega, review y retro (comparten tiempo con el Sprint 4):

- **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub: (1) en Colab, *Archivo → Descargar → Descargar .ipynb* (después de *Reiniciar y ejecutar todo*) y descargad también `turisdata_reto.db` desde el panel de archivos; (2) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-05`**; (3) *Add file → Upload files* y arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace: repartid las subidas entre personas (por ejemplo, una sube el cuaderno y otra la base de datos). Nombres de archivo sin espacios ni tildes. El CSV va a `data/` solo si aún no está.
- **La autoevaluación.** Cada persona rellena y sube la suya a `sprint-05/`; la coevaluación va dentro del mismo fichero.
- **La review (5 minutos por equipo).** Enseñad: (1) la pregunta y la base de datos; (2) cómo converge vuestro descenso del gradiente y su comparación con la librería; (3) el resultado clave con su intervalo; (4) cómo habéis distinguido lo que los datos muestran de lo que no pueden demostrar; (5) la recomendación al cliente.

Qué deberíais ver. En la carpeta `sprint-05` del repositorio, el cuaderno, la base de datos y una hoja de autoevaluación por persona (**2 + N**), con *commits* de personas distintas en el historial (el icono del reloj).

Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código y explicaciones que parecen correctos y no lo son**. Reglas:

- **El descenso del gradiente lo escribís vosotros.** Si se lo pedís a la IA, no podréis explicarlo en la review.
- **Cualquier cifra** que aparezca en la respuesta tiene que salir de vuestro cuaderno, no de la IA.
- **Declarad** en la celda «Uso de IA» qué pedisteis y qué comprobasteis.
- **Nada de datos personales ni claves** en una IA.

Para qué	Ejemplo de petición	Qué comprobáis
Entender una idea	«Explícame el descenso del gradiente como si fuera bajar una montaña con niebla, sin fórmulas»	Que podéis explicarlo sin mirar
Entender un error de código	«Mi curva de aprendizaje sube en vez de bajar. Este es mi bucle: *[código]*. ¿Qué puede pasar?»	Que probáis la sugerencia y la curva baja
Interpretar un resultado	«Tengo un p-valor de 0,03 y un intervalo de confianza de [−1,1; −0,4]. Explícamelo para un directivo, con un ejemplo»	Que la explicación coincide con **vuestros** números
Revisar la redacción	«Revisa mi respuesta de 150 palabras: ¿hay jerga o algo ambiguo? No cambies las cifras»	Que las cifras siguen siendo las vuestras

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
La curva de aprendizaje sube o da `NaN`	Tasa de aprendizaje demasiado grande o variables sin estandarizar	Reducir la tasa y estandarizar la temperatura
Vuestra recta no coincide con `statsmodels`	Pocas épocas o tasa muy pequeña	Más épocas o ajustar la tasa
El punto de control 1 falla por el índice o por `con`	El índice no se llama `idx_ocupacion_fecha` o la conexión no se llama `con`	Usar exactamente esos nombres
Error al repetir la celda de la base	Falta `if_exists="replace"` al guardar la tabla	Añadirlo
Decir que «el calor provoca…»	Se confunde correlación con causa	Hablar de «se asocia con» y explicar los límites
Olvidar la estacionalidad	El verano coincide con calor Y con vacaciones	Incluir `mes` en el modelo con control
Dar solo el p-valor	«Es lo que importa»	Dar también el **tamaño** del efecto y el **intervalo**
El cuaderno solo funciona en un orden raro	Se ejecutaron celdas sueltas	*Reiniciar y ejecutar todo* antes de entregar
La respuesta pasa de 150 palabras	No se ha contado	`len(texto.split())` y recortar

Ayuda 3 · Si os atascáis

1. Leed el mensaje del `assert` que falla: dice qué esperaba y qué recibió. Probad la función con un ejemplo pequeño en una celda aparte.
2. Repasad el cuaderno de esa parte: `S05_03` para el gradiente, `S05_05` para `statsmodels` y los p-valores, `S05_06` para la base de datos; y el código explicado.
3. Preguntad al equipo y, después, al docente (no más de 15 minutos atascados).